

DOM Basics in JavaScript

In the previous lessons, we studied the basic features of the JavaScript language. Starting with this lesson, we will do what JavaScript itself is intended for - we will change the elements of the HTML page and respond to user actions. Our scripts will become more spectacular and useful.

In all of the following tutorials, your JavaScript code must be placed below the HTML code it refers to. That is, the structure of your code should look like this:

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4
5    </head>
6    <body>
7      your html tags
8
9      <script>
10       your JavaScript code
11     </script>
12   </body>
13 </html>
```

Or as follows, if you write JavaScript code in a separate file:

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4
5    </head>
6    <body>
7      your html tags
8
9      <script src="index.js"></script>
10   </body>
11 </html>
```

In the future, for brevity, I will not give the full code, but I will simply indicate what needs to be written in the HTML part, and what - in the JavaScript part.

№ 1

©jsPmDmInr

Make a file with the specified code structure.

DOM elements in JavaScript

Some object with its own properties corresponds to each page tag in JavaScript. This object stores the text of the tag, the values of its attributes, and other useful things. In JavaScript, such objects are called *DOM elements*.

Let our HTML code have the following tag:

```
1 <div id="elem">text</div>
```

Let the variable `elem` contain a reference to this tag. In this case, the variable `elem` will be an object with properties `id` and `textContent`. The `id` property will contain the value of the `id` attribute of our tag, and the `textContent` property will contain its text.

In the next lesson, we will learn how to get references to page tags in JavaScript.

Getting the DOM element in JavaScript

Now we will learn how to get the page DOM elements in order to perform some manipulations with them in the future.

Let's have some button:

```
1 <input id="button" type="submit">
```

Let's get a reference to this button into a variable. To do this, use the `querySelector` method of the special object `document`. This method takes a CSS selector as a parameter and returns a reference to the element found with this selector.

Our button has an attribute `id` with the value `button`. So we can find this button by the selector `#button`. So, let's find our button and write a reference to it into a variable:

```
1 let button = document.querySelector('#button');
2 console.log(button);
```

№ 1

⊗jsPmDmEG

Given 3 paragraphs:

```
1 <p id="elem1">1</p>
2 <p id="elem2">2</p>
3 <p id="elem3">3</p>
```

Put the reference to each of the paragraphs in a separate variable and print the contents of each of these variables to the console.

Advanced selectors of the DOM element in JavaScript

Let's do something more complicated. Let, for example, we have the following HTML code:

```
1 <div id="parent">
2   <input>
3 </div>
```

Let's get a reference to the input located inside the block `#parent`:

```
1 let elem = document.querySelector('#parent input');
2 console.log(elem);
```

As you can see, we got our input using the `#parent input` selector. In fact, all inputs that are in `#parent` match this CSS selector.

Let, for example, we have two such inputs:

```
1 <div id="parent">
2   <input value="1">
3   <input value="2">
4 </div>
```

Then, if we write the selector `#parent input` in the CSS code, it will affect both of our inputs:

```
1 #parent input {
2   color: red;
3 }
```

The `querySelector` method, however, does not work that way. It always gets only one element - the first one that matches the specified selector. Let's look at an example. Let's say we have inputs with class `elem`:

```
1 <input class="elem">
2 <input class="elem">
```

Let's get the first of these inputs:

```
1 let elem = document.querySelector('.elem');
2 console.log(elem); // here will be the first input
```

№1

⊗jsPmDmEGAS

Given the following HTML:

```
1 <div id="block">
2   <p>1</p>
3   <p>2</p>
4 </div>
```

Get a reference to the first paragraph from a div with `id` equal to `block`.

№2

⊗jsPmDmEGAS

Given the following HTML:

```
1 <div class="block">
2   <p>1</p>
3   <p>2</p>
4 </div>
```

Get a reference to the first paragraph from a div with the class `block`.

Given the following HTML:

```
1 <p class="www">text</p>
2 <p class="www">text</p>
```

Get a reference to the first paragraph with the class `www`.

Attaching handlers to elements in JavaScript

Let's now teach our DOM elements to respond to site user actions. For example, the user clicks somewhere with the mouse, and our code in response will have to process this click and display some information on the screen.

User actions that we can track through JavaScript are called *events*. Events can be the following: a mouse click on a page element, mouse hover over a page element, or vice versa - the mouse cursor leaves the element, and so on. In addition, there are events that do not depend on user actions, for example, an event when an HTML page is loaded into the browser.

For example, let's make a button, by clicking on which some text will be displayed on the screen. Let's start with the HTML code for the button:

```
1 <input id="button" type="submit">
```

Now we get a reference to the button into a variable:

```
1 let button = document.querySelector('#button');
```

Now we need to set the reaction of our button when it is clicked. To do this, JavaScript has a special method `addEventListener`, which takes the name of the event as the first parameter (clicking on the button is called 'click'), and the second parameter is a *callback function* that is executed when this event occurs.

Let's, for example, by clicking on the button display some text:

```
1 button.addEventListener('click', function() {
2   console.log('!!!');
3 });
```

Given 3 buttons:

```
1 <input id="button1" type="submit">
2 <input id="button2" type="submit">
3 <input id="button3" type="submit">
```

Make it so that clicking on the first button displays the number 1, clicking on the second button displays the number 2, and clicking on the third button displays the number 3.

Named event handlers in JavaScript

In the previous lessons, we used anonymous functions as event handlers. This is actually not necessary - the function can be ordinary, with a name. For example, let's say we have the following function:

```
1 function func() {  
2   console.log('!!!');  
3 }
```

Let's also have a button:

```
1 <input id="button" type="submit">
```

Let's make it so that when the button is clicked, our function `func` is executed. To do this, with a parameter of `addEventListener` we will pass the name of our function, like this:

```
1 let button = document.querySelector('#button');  
2 button.addEventListener('click', func);  
3  
4 function func() {  
5   console.log('!!!');  
6 }
```

№1

⊗jsPmDmNEH

Given the following HTML code:

```
1 <input id="button1" type="submit" value="button1">  
2 <input id="button2" type="submit" value="button2">
```

Given the following functions:

```
1 function func1() {  
2   console.log(1);  
3 }  
4  
5 function func2() {  
6   console.log(2);  
7 }
```

Make it so that when the first button is clicked, the function `func1` is executed, and when the second button is clicked, the function `func2` is executed.

Single handler for multiple elements in JavaScript

A single function can be bound to multiple elements at once. For example, let's say we have the following function:

```
1 function func() {  
2   console.log('!!!');  
3 }
```

There are also two buttons:

```
1 <input id="button1" type="submit" value="button1">
2 <input id="button2" type="submit" value="button2">
```

Get references to these buttons into variables:

```
1 let button1 = document.querySelector('#button1');
2 let button2 = document.querySelector('#button2');
```

And now let's bind our function `func` to both the first and the second button:

```
1 button1.addEventListener('click', func);
2 button2.addEventListener('click', func);
```

№ 1

⊗jsPmDmMEH

Given 5 paragraphs:

```
1 <p id="elem1">text</p>
2 <p id="elem2">text</p>
3 <p id="elem3">text</p>
4 <p id="elem4">text</p>
5 <p id="elem5">text</p>
```

Given the following function:

```
1 function func() {
2   console.log('message');
3 }
```

Bind this function to all 5 paragraphs.

Multiple handlers for one event in JavaScript

Multiple functions can be attached to one element at once. Let's look at an example. Let's have a button:

```
1 <input id="button" type="submit">
```

Let's say we have two functions:

```
1 function func1() {
2   console.log('1');
3 }
4
5 function func2() {
6   console.log('2');
7 }
```

Get a link to our button into a variable:

```
1 let button = document.querySelector('#button');
```

And now let's bind both the first and second functions to our button as click handlers:

```
1 button.addEventListener('click', func1);
2 button.addEventListener('click', func2);
```

№ 1

⊗jsPmDmSH

Given a paragraph:

```
1 <p id="elem">text</p>
```

Given the following functions:

```
1 function func1() {
2   console.log('1');
3 }
4
5 function func2() {
6   console.log('2');
7 }
8
9 function func3() {
10  console.log('3');
11 }
```

Bind all these functions to our paragraph.

Handlers for different events in JavaScript

In addition to clicking on an element, there are other events. For example, using the `dblclick` event, you can catch a double click on an element, using the `mouseover` event - hovering over the element, and using the `mouseout` event - leaving the cursor from the element.

At the same time, handlers of various types of events can be attached to one element. Let's, for example, bind a cursor hover response and a leave response to the same element:

```
1 button.addEventListener('mouseover', function() {
2   console.log('1');
3 });
4 button.addEventListener('mouseout', function() {
5   console.log('2');
6 });
```

№ 1

⊗jsPmDmDEH

Given a button. By double-clicking on it, display some message.

№ 2

⊗jsPmDmDEH

Given a button. Display a message by hovering over it.

№3

⊗jsPmDmDEH

Given a button. When the cursor leaves it, display some message.

№4

⊗jsPmDmDEH

Given a button. Display one message when you hover over it, and another message when you leave it.

Working with element text in JavaScript

DOM elements have a `textContent` property that allows you to read the text of those elements. Let's look at an example. Let's say we have the following tag:

```
1 <p id="elem">text</p>
```

Get a reference to this tag into a variable:

```
1 let elem = document.querySelector('#elem');
```

Read the tag text:

```
1 console.log(elem.textContent);
```

Change the tag text:

```
1 elem.textContent = '!!!!';
```

№1

⊗jsPmDmET

Given a paragraph and a button. By clicking on the button, read the text of the paragraph and output it to the console.

№2

⊗jsPmDmET

Given a paragraph and a button. By clicking on the button, write a new text into the paragraph.

№3

⊗jsPmDmET

Given two paragraphs containing some numbers in their text, and a button. By pressing the button, display the sum of the stored numbers.

№4

⊗jsPmDmET

Given three paragraphs with numbers, a div, and a button. By pressing the button, write down the sum of numbers in the text of the div.

№5

⊗jsPmDmET

Given a paragraph with a number and a button. By clicking on the button, add one to the paragraph value and write the resulting number back.

№6

⊗jsPmDmET

Given a paragraph with text and a button. After pressing the button, write an exclamation mark at the end of the paragraph text.

Working with HTML code of element in JavaScript

DOM elements have an `innerHTML` property that allows you to read the HTML code of those elements. Let's look at an example. Let's say we have the following tag:

```
1 | <p id="elem"><b>text</b></p>
```

Get a reference to this tag into a variable:

```
1 | let elem = document.querySelector('#elem');
```

Read the HTML code of the tag:

```
1 | console.log(elem.innerHTML); // shows <b>text</b>
```

Change the tag text:

```
1 | elem.innerHTML = '<i>!!!</i>';
```

№1

⊗jsPmDmEH

Given a paragraph and a button. On button click, read the HTML code of the paragraph and output it to the console.

No 2

⊗jsPmDmEH

Given a paragraph and a button. By clicking on the button, enter new text into the paragraph so that it is bold.

Tag attributes as properties in JavaScript

Let's now learn how to get tag attributes. Here the following rule applies: *each attribute of the tag corresponds to the DOM element's property of the same name.*

Let's look at an example. Let's say we have this tag:

```
1 | <input id="elem" type="text">
```

Get a reference to our element into a variable:

```
1 | let elem = document.querySelector('#elem');
```

Derive the values of the attributes we need:

```
1 | console.log(elem.id); // shows 'elem'
2 | console.log(elem.type); // shows 'text'
```

And now let's change the value of the attribute:

```
1 | elem.type = 'submit';
```

No 1

⊗jsPmDmAVP

Given the following input:

```
1 | <input id="elem" type="email">
```

There is also a button. On clicking the button, display the contents of the attribute **type** of the above input.

No 2

⊗jsPmDmAVP

Given the following input:

```
1 | <input id="elem" type="email">
```

There is also a button. When the button is clicked, write the value **submit** to the **type** attribute.

No 3

⊗jsPmDmAVP

Suppose you have a link in the form of a tag **a**, a button and a paragraph. On clicking the button, output the contents of the link attribute **href** into the paragraph.

№4

⊗ jsPmDmAVP

Suppose you have a link and a button. On clicking the button, add the contents of its attribute `href` to the end of the link text in parentheses.

№5

⊗ jsPmDmAVP

Let's say you have a picture. Place it on the page using the tag `img`. Let a button and a paragraph be given as well. Make it so that when you click on the button, the path to the image from its attribute `src` is written to the paragraph.

№6

⊗ jsPmDmAVP

Given a picture in the `img` tag and a button. By pressing the button write down the value 300 in the attribute `width`.

№7

⊗ jsPmDmAVP

Given a picture in the `img` tag and a button. Let the attribute `width` be set to some width. Make a button, upon clicking on which the width of the image will increase by 2 times.

№8

⊗ jsPmDmAVP

Let's say you have two pictures. Make an `img` tag and two buttons on the page. On pressing the first button, write the path to the first picture in the `src` attribute, and on pressing the second button, write the path to the second picture.

Working with text fields in JavaScript

Now we will learn how to get text from users of our site. To do this, HTML provides a special tag `input`, which is a text field for data entry.

```
1 | <input>
```

This tag has the special `value` attribute that specifies the initial text that will be written in the input when entering the page:

```
1 | <input value="text">
```

The user of our site after entering the page can change the text of the input. Moreover, if we have a variable containing a reference to this input, then the `value` property of this variable will always contain the current value of the input text.

Let's try it in practice. Let us have an input with the attribute `value`:

```
1 <input id="elem" value="text">
```

Get a reference to this input in a variable:

```
1 let elem = document.querySelector('#elem');
```

And now we will display the current input text:

```
1 console.log(elem.value);
```

And now change the text of the input to another:

```
1 elem.value = 'new text';
```

№ 1

⊗ jsPmDmTF

Given an input and a button. On pressing the button, write some text to the input.

№ 2

⊗ jsPmDmTF

Given an input, a paragraph, and a button. By pressing the button, write the text from the input into the paragraph.

№ 3

⊗ jsPmDmTF

Given two inputs and a button. The user enters a number into the first input. By pressing the button, write the square of the entered number into the second input.

№ 4

⊗ jsPmDmTF

Given two inputs and a button. By pressing the button, write the value of the second input into the first input, and the value of the first input into the second input. Your code should work universally, for any input values.

№ 5

⊗ jsPmDmTF

Given 5 inputs, a paragraph, and a button. Numbers are entered into the inputs. By pressing the button, write down the arithmetic mean of the entered numbers into a paragraph.

Text fields focus in JavaScript

Let's have an input. You can click on this input and after that the text cursor will blink in it and you will be able to enter text into it.

This state is said to be that the input now has *input focus*. In practice, this means that if you start typing text from the keyboard, then this text will fall into the input that currently has the input focus. If you then click anywhere outside of the input, that input *will lose the focus* and you won't be able to type into it.

In order to catch the moment an input gains or loses focus, JavaScript provides special events: the `focus` event allows you to catch the focus gain by an input, and the `blur` event catches the loss. Let's try it in practice. Let us have an input:

```
1 <input id="elem" value="text">
```

Let's get a reference to it into a variable:

```
1 let elem = document.querySelector('#elem');
```

And now let's make it so that when focus is received, the current input text is displayed in the console:

```
1 elem.addEventListener('focus', function() {  
2   console.log(elem.value);  
3 });
```

№1

⊗ jsPmDmTFF

Given an input. When focus is gained, write the number 1 into it, and when the focus is lost, write the number 2.

№2

⊗ jsPmDmTFF

An input is given. Let's put in a number. When focus is lost, display the square of this number.

№3

⊗ jsPmDmTFF

An input is given, in which there is some text initially. When an input receives focus, clear the contents of that input.

Exceptions when working with attributes in JavaScript

There is an exception when working with attributes - this is the attribute `class`. This word is special in JavaScript and therefore we cannot simply write `elem.class` to read the value of the `class` attribute. Instead, write `elem.className`.

Let's look at an example. Let's say we have this input:

```
1 <input id="elem" class="aaa bbb">
```

Let's output the value of the `class` attribute for our input:

```
1 let elem = document.querySelector('#elem');
2 console.log(elem.className); // shows 'aaa bbb'
```

There are other exceptions, such as the `for` attribute. It should be accessed via `htmlFor`.

No 1

⊗ jsPmDmEA

A div is given:

```
1 <div id="elem" class="content no-gap"></div>
```

There is also a button. By clicking on the button, read and display the value of the `class` attribute of our div.

No 2

⊗ jsPmDmEA

A div is given:

```
1 <div id="elem"></div>
```

There is also a button. By pressing the button, write some class into the `class` attribute of our div.

No 3

⊗ jsPmDmEA

Given a div with several CSS classes separated by spaces:

```
1 <div id="elem" class="aaa bbb ccc"></div>
```

There is also a button. On button click, get an array of CSS classes of our div.

Methods and properties chaining in JavaScript

Let us have an input:

```
1 <input id="elem" value="text">
```

Let's display the input text:

```
1 let elem = document.querySelector('#elem');
2 console.log(elem.value); // shows 'text'
```

As you can see, we first get an element by its `id`, write this element to the variable `elem`, and then display the property `value` from this variable

In fact, you can not enter the variable `elem`, but build a chain of dots in this way:

```
1 console.log( document.querySelector('#elem').value ); // shows 'text'
```

In the same way - in a chain - you can overwrite attributes:

```
1 document.querySelector('#elem').value = 'www';
```

№ 1

⊗jsPmDmChn

Given the following code:

```
1 

1 let image = document.querySelector('#image');
2 console.log(image.src);
```

Modify the code above to use a chain instead of introducing the variable `image`.

Advantages and disadvantages of chains in JavaScript

Despite the fact that the chains reduce the code, in most cases the introduction of a variable is still more convenient. Compare the two examples - now I have entered the variable `elem` and I can write any number of attributes, while `querySelector` is called only once:

```
1 let elem = document.querySelector('#elem');
2 elem.value = 'www';
3 elem.type = 'submit';
```

And now I'm not introducing a new variable and so I have to call `querySelector` twice:

```
1 document.querySelector('#elem').value = 'www';
2 document.querySelector('#elem').type = 'submit';
```

In my opinion, this code has become more complicated, although it takes one line less. Also, if I want to change the value of `id` from 'elem' to something else, I have to do it in many places, which is not very convenient.

There is also another problem - the load on the browser. Searching for elements on the page, which is done by the `querySelector` method, is a rather slow operation (*and in general, any work with page elements is a slow operation* - remember this).

In our case, if we use `querySelector` every time, then the browser will process the HTML page each time and look for an element with a given `id` several times (it doesn't matter that `id` are the same - the browser will do all the actions several times), performing useless operations that can slow down the browser.

If we use the variable `elem` - no search occurs on the page (the element has already been found and the reference to it lies in the variable).

№ 1

⊗jsPmDmChA

Given the following code:

```
1 

1 console.log(document.querySelector('#image').src);
2 console.log(document.querySelector('#image').width);
3 console.log(document.querySelector('#image').height);
```

Point out the disadvantages of this code. Remove them.

this object in JavaScript

Now we will work with a special object `this` available in the event handler function. This object points to the element where the event occurred.

The `this` object is useful when the element on which the event occurred and the element being acted upon as a result of the event are the same element.

For example, if we have an input, we can attach a focus loss handler to it and, upon the occurrence of this event, do something with the input text. Let's do the above. Let us have an input:

```
1 <input id="elem" value="text">
```

Let's get a reference to it into the variable `elem`:

```
1 let elem = document.querySelector('#elem');
```

Let's bind the `blur` event handler function to it:

```
1 elem.addEventListener('blur', func);
```

Inside this function `func`, an object `this` will be available referring to our input:

```
1 function func() {  
2   console.log(this); // contains a reference to our element  
3 }
```

Let's output the content of the `value` attribute of our input:

```
1 function func() {  
2   console.log(this.value); // displays the content of the attribute  
3 }
```

Well, now let's write some text to the input:

```
1 function func() {  
2   this.value = '!!!';  
3 }
```

You can also use an anonymous function:

```
1 elem.addEventListener('blur', function() {  
2   this.value = '!!!';  
3 });
```

№1

⊗jsPmDmThs

Input is given. When this input receives focus, write the number 1 into it, and when it loses focus, write the number 2. To refer to an input inside a handler function, use the `this` object.

№2

⊗jsPmDmThs

Given a button whose value is the number 1. Make it so that when you click on this button, its value increases by one each time.

this object advantage in JavaScript

From what has been said before, the special advantage of `this` is not yet obvious. Indeed, inside the handler function, our element will be available anyway - after all, the variable `elem` will be global for our function `func`:

```
1 let elem = document.querySelector('#elem');
2 elem.addEventListener('click', func);
3
4 function func() {
5   // elem variable with our element is available here
6 }
```

And, it is easy to figure out that the content of `this` and the content of the variable `elem` in our case are equal:

```
1 let elem = document.querySelector('#elem');
2 elem.addEventListener('click', func);
3
4 function func() {
5   console.log(elem === this); // shows true
6 }
```

What is the special advantage of `this`? It manifests itself when we have multiple elements, and the same function is attached to each.

Let's look at an example. Let's say we have 3 buttons:

```
1 <input id="button1" type="submit" value="text1">
2 <input id="button2" type="submit" value="text2">
3 <input id="button3" type="submit" value="text3">
```

Get references to them in variables:

```
1 let button1 = document.querySelector('#button1');
2 let button2 = document.querySelector('#button2');
3 let button3 = document.querySelector('#button3');
```

Bind the same function to these buttons:

```
1 button1.addEventListener('click', func);
2 button2.addEventListener('click', func);
3 button3.addEventListener('click', func);
```

And inside the function we will display `this.value`:

```
1 function func() {
2   console.log(this.value);
3 }
```

It turns out that we have three buttons. Pressing each button will call the function `func`. In this case, with each click, `this` will contain a reference to the button in which the event occurred.

That is, each press will output to the console `value` of the button that was pressed, but this will be done by the same function `func`! That's the advantage of using `this`.

№ 1

⊗jsPmDmThsA

Given 5 paragraphs with some texts. By clicking on any paragraph, write an exclamation mark at the end of its text.

№2

⊗jsPmDmThsA

Given 3 inputs, in which some numbers are written. On loss of focus in any of the inputs, square the number standing in it.

Getting a group of elements in JavaScript

In the previous lessons, we used the `querySelector` method to get one element on page. Now it's time to learn how to get a group of elements and do some operations with many elements at once.

To do this, there is a `querySelectorAll` method that receives all tags that match the CSS selector as an array of elements. To do something with the found elements, you need to work with the resulting array, for example, loop through it and perform some operation with each element in the loop separately.

Let, for example, we have paragraphs with the class `www`:

```
1 <p class="www">text1</p>
2 <p class="www">text2</p>
3 <p class="www">text3</p>
```

Let's get an array of these paragraphs, loop through them and in the loop print the texts of the found paragraphs to the console:

```
1 let elems = document.querySelectorAll('.www');
2
3 for (let elem of elems) {
4   console.log(elem.textContent);
5 }
```

And now let's add an exclamation mark to the end of each paragraph text:

```
1 let elems = document.querySelectorAll('.www');
2
3 for (let elem of elems) {
4   elem.textContent = elem.textContent + '!';
5 }
```

№1

⊗jsPmDmEGG

Given paragraphs and a button. On button click, find all paragraphs, loop through and set the text of each paragraph to 'text'.

№2

⊗jsPmDmEGG

Given paragraphs with text and a button. By clicking on the button, write down its ordinal number at the end of the text of each paragraph.

№3

⊗jsPmDmEGG

Given inputs with numbers, a paragraph and a button. By pressing the button, find the sum of numbers from the inputs and write this sum in the text of the paragraph.

Adding handlers in a loop in JavaScript

Let's now learn how to bulk add event handlers to elements. Let, for example, we have paragraphs:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Let's also have a function:

```
1 function func() {
2   console.log('!');
3 }
```

Let's iterate over our paragraphs in a loop and add the function `func` as a click handler to each paragraph:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', func);
5 }
```

Let's go ahead and make it so that when you click on any paragraph, the text of this paragraph is displayed. There is, however, a problem: there are many paragraphs, and there is only one handler function. How do we distinguish our paragraphs inside the handler function?

The `this` object will help us with it - when the function is called at the time of the event, this object will point to the element where this event happened. Let's rewrite the code of our function `func` in accordance with the above:

```
1 function func() {
2   console.log(this.textContent); // outputs the paragraph text
3 }
```

№1

⊗jsPmDmLNH

Given the following function:

```
1 function func() {
2   this.value = Number(this.value) + 1;
3 }
```

Inputs are also given. Make it so that on loss of focus in any of our inputs, the above function is executed.

№2

⊗jsPmDmLNH

Given paragraphs with numbers. Make it so that when you click on any paragraph, its number in it is squared.

Adding anonymous handlers in a loop in JavaScript

Anonymous functions can also be used as handlers added in a loop. This makes the code more compact and saves us from having to come up with a name for a function that is only used in one place.

Let's add anonymous handlers to elements:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     console.log(this.textContent);
6   });
7 }
```

No 1

⊗jsPmDmLAH

Given the following code:

```
1 <div>1</div>
2 <div>2</div>
3 <div>3</div>
4 <div>4</div>
5 <div>5</div>

1 let divs = document.querySelectorAll('div');
2
3 for (let div of divs) {
4   div.addEventListener('click', func);
5 }
6
7 function func() {
8   this.textContent++;
9 }
```

Make the handler function anonymous.

Unbinding events in JavaScript

In this lesson, we will learn how to unbind event handlers that we previously attached to elements. Let the following button be given as an example:

```
1 <input id="button" type="submit">
```

Bind the function `func` to this button:

```
1 let button = document.querySelector('#button');
2 button.addEventListener('click', func);
3
4 function func() {
5   console.log('!!!');
6 }
```

Let's now make the event handler fire on the first click, and then unbind from the button. There is a special method `removeEventListener` for this. This method takes the type of the event as the first parameter, and the reference to the function to be unbindable as the second parameter.

Typically, this means that the event handler is unbound the same way it was bound. That is, if we bind it like this: `addEventListener('click', func)`, then we will unbind it with the same parameters, like this: `removeEventListener('click', func)` [/ c].

So let's solve our problem:

```
1 let button = document.querySelector('#button');
2 button.addEventListener('click', func);
3
4 function func() {
5     console.log('!!!');
6     this.removeEventListener('click', func);
7 }
```

№1

⊗jsPmDmHU

Given a link and a button. On button click add to the end of the link text the value of its href attribute in parentheses. Make it so that this addition occurs only on the first click.

№2

⊗jsPmDmHU

Given a button whose value is the number 1. Make it so that when you click on this button, its value increases by one each time. After the button's value reaches 10 - unbind the event handler so that the button no longer responds to pressing.

Unbinding event handlers in a loop in JavaScript

Let now we have not single element, but several. For example, a few paragraphs:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Let's bind the function `func` as click handler to each of these paragraphs:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4     elem.addEventListener('click', func);
5 }
6
7 function func() {
8     console.log(this.textContent);
9 }
```

Let's now rewrite the code so that each paragraph only responds to the first click on it. To do this, when clicking on a paragraph, we will unbind the attached handler from it. In this case, the unbinding will be specifically from this paragraph, without affecting the rest.

As you already know, the element in which the event occurred can be accessed in the handler function via `this`. This means that you need to unbind the handler from `this`, like this:

```

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', func);
5 }
6
7 function func() {
8   console.log(this.textContent);
9   this.removeEventListener('click', func); // unbinds the handler
10 }

```

№ 1

©jsPmDmLHU

Given paragraphs. By clicking on any of the paragraphs, add an exclamation mark to the end of it. Make it so that this addition occurs only on the first click.

Unbinding anonymous functions in JavaScript

Let now an anonymous function is attached to our paragraphs:

```

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     console.log(this.textContent);
6   });
7 }

```

Suppose we want to unbind this function from a paragraph after clicking on this paragraph. However, we are faced with a problem: the function does not have a name, which means we will not be able to refer to it by this name in order to unbind it.

To solve the problem, you need to give the name to the function, making it **named functional expression**. Let's do it:

```

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function func() { // gives the function name
5     console.log(this.textContent);
6   });
7 }

```

Now this function can be unbind within itself:

```

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function func() {
5     console.log(this.textContent);
6     this.removeEventListener('click', func); // unbinds the function
7   });
8 }

```

№ 1

©jsPmDmAHU

Given the list **ul**, each item of which contains a number. Make it so that when you click on any **li**, its number increases by one.

№2

⊗jsPmDmAHU

Modify the previous task so that each `li` only increases its value the first time it is clicked.

№3

⊗jsPmDmAHU

Modify the previous task so that each `li` increases its value only if its value is less than 10.

DOM Coding Tips in JavaScript

Suppose you are faced with a task of sufficient complexity, for the implementation of which you need to write a certain number of code lines.

The wrong approach would be to try to write the entire code of the solution as a whole, and then start testing it. In this case, there is a high probability that nothing will work for you, and you will have to look for the error in a large amount of code.

The correct approach is to break the task into small elementary steps that you will implement and immediately check their correctness. In this case, even if you make a mistake somewhere, you will immediately notice the problem and fix it.

Let's try it in practice. Let's say you have paragraphs:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>
4 <p>4</p>
5 <p>5</p>
6 <p>6</p>
```

Let your task be to find paragraphs with numbers divisible by 3 and find a sum of those numbers.

As a first small step, I would suggest getting our paragraphs as an array and printing this array to the console to see if we got everything right. Let's do it:

```
1 let elems = document.querySelectorAll('p');
2 console.log(elems);
```

The next small step is to loop through our paragraphs and output each of them to the console individually:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   console.log(elem);
5 }
```

And now in a loop we will display the texts of our paragraphs:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   console.log(elem.textContent);
5 }
```

Now let's display the texts of those paragraphs whose number is divisible by 3:

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   let text = +elem.textContent;
5
6   if (text % 3 === 0) {
7     console.log(text);
8   }
9 }
```

After making sure that we get the correct paragraphs, we can proceed to sum their numbers:

```
1 let elems = document.querySelectorAll('p');
2 let sum = 0;
3
4 for (let elem of elems) {
5   let text = +elem.textContent;
6
7   if (text % 3 === 0) {
8     sum += text;
9   }
10 }
11
12 console.log(sum);
```

№ 1

⊗jsPmDmCd g

Given a list containing the years:

```
1 <ul>
2   <li>2000</li>
3   <li>2004</li>
4   <li>2021</li>
5   <li>2022</li>
6   <li>2025</li>
7   <li>2031</li>
8 </ul>
```

Get years whose sum of digits is 6. Find the sum of the years obtained.

Finding errors in code with DOM in JavaScript

In the following tasks, some programmer has written code and may have made mistakes in it. You should check if the code does what is described. If the code does not work correctly, you need fix the errors.

№ 1

⊗jsPmDmMst

The code should add text to the end of each paragraph:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>

1 let elems = document.querySelector('p');
2 elems.textContent += '!!';
```

№2

⊗jsPmDmMst

When you click on a paragraph, its value should increase by one:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function() {
5     this.textContent += Number(1);
6   });
7 }
```

№3

⊗jsPmDmMst

By clicking on the button, the text of the paragraph should become bold:

```
1 <p>text</p>
2 <button>click</button>

1 let button = document.querySelector('button');
2 let elem = document.querySelector('p');
3
4 button.addEventListener('click', function() {
5   elem.innerHTML = '<b>elem.innerHTML</b>';
6 });
```

№4

⊗jsPmDmMst

By clicking on the button, the sum of the numbers from the paragraphs should be displayed:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>
4
5 <button>click</button>

1 let button = document.querySelector('button');
2 let elems = document.querySelectorAll('p');
3
4 button.addEventListener('click', function() {
5   let sum = 0;
6
7   for (let elem of elems) {
8     sum += Number(elem.textContent);
9   }
10
11   console.log(sum);
12 });
```

№5

⊗jsPmDmMst

By clicking on a paragraph, the specified text should be added to the end of it:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>

1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', () => {
5     this.textContent += '!';
6   });
7 }
```

№6

⊗jsPmDmMst

By clicking on the button, the text of each paragraph should become bold:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
4 <button>click</button>

1 let button = document.querySelector('button');
2 let elems = document.querySelectorAll('p');
3
4 button.addEventListener('click', function() {
5   for (let elem of elems) {
6     elem.innerHTML = '<b>'+elem.innerHTML+'</b>';
7   }
8 });
```

№7

⊗jsPmDmMst

By clicking on the button, the sum of the numbers from the paragraphs should be displayed in the console:

```
1 <p>1</p>
2 <p>2</p>
3 <p>3</p>
4
5 <button>click</button>

1 let button = document.querySelector('button');
2 let elems = document.querySelectorAll('p');
3 let sum = 0;
4
5 for (let elem of elems) {
6   sum = elem.textContent + 1;
7
8   button.addEventListener('click', function() {
9     console.log(sum);
10   });
11 }
```

№8

⊗ jsPmDmMst

Numbers are entered into the inputs. By clicking on the button, the sum of the numbers from the inputs should be displayed in the console:

```
1 <input>
2 <input>
3 <input>
4
5 <button>click</button>

1 let button = document.querySelector('button');
2 let elems = document.querySelectorAll('input');
3 let sum = 0;
4
5 for (let elem of elems) {
6   sum += elem.value;
7 }
8
9 button.addEventListener('click', function() {
10   console.log(sum);
11 });
```

№9

⊗ jsPmDmMst

Numbers are entered into the first two inputs. By clicking on the button, the sum of these numbers should be displayed in the third input:

```
1 <input id="inp1">
2 <input id="inp2">
3 <input id="inp3">
4
5 <button id="btn">click</button>

1 let btn = document.querySelector('#btn');
2 let inp1 = document.querySelector('#inp1');
3 let inp2 = document.querySelector('#inp2');
4 let inp3 = document.querySelector('#inp3');
5
6 btn.addEventListener('click', function() {
7   inp3.textContent = inp1.textContent + inp2.textContent;
8 });
```

№10

⊗jsPmDmMst

Numbers are entered into the first two inputs. By clicking on the button, the sum of these numbers should be displayed in the paragraph:

```
1 <input id="inp1">
2 <input id="inp2">
3 <p id="res"></p>
4
5 <button id="btn">click</button>

1 let btn = document.querySelector('btn');
2 let res = document.querySelector('res');
3 let inp1 = document.querySelector('inp1');
4 let inp2 = document.querySelector('inp2');
5
6 button.addEventListener('click', function() {
7   res.value = inp1.value + inp2.value;
8 });
```

№11

⊗jsPmDmMst

On button click, you need to check that the text in each input matches the text of its data attribute:

```
1 <input data-text="text1">
2 <input data-text="text2">
3 <input data-text="text3">
4
5 <button id="btn">click</button>

1 let inputs = document.querySelectorAll('input');
2 let button = document.querySelector('#button');
3
4 button.addEventListener('click', function() {
5   for (let input of inputs) {
6     if (input.value === input.dataset) {
7       input.classList.add('right')
8     } else {
9       input.classList.add('wrong')
10    }
11  }
12 });
```

№12

⊗jsPmDmMst

By clicking on the button, you need to check that the text in each input matches the text of the corresponding array element:

```
1 <input>
2 <input>
3 <input>
4
5 <button id="btn">click</button>

1 let inputs = document.querySelectorAll('input')
2 let button = document.querySelector('#button')
3
4 let texts = [
5   'text1',
6   'text2',
7   'text3',
8 ];
9
10 button.addEventListener('click', function() {
11   for (let input of inputs) {
12     for (let text of texts) {
13       if (input.value === text) {
14         input.classList.add('right')
15       } else {
16         input.classList.add('wrong')
17       }
18     }
19   }
20 });
```

№13

⊗jsPmDmMst

Numbers are entered into the inputs. By clicking on the button, the sum of the entered numbers should be displayed in the console:

```
1 <input>
2 <input>
3 <input>
4
5 <button id="btn">click</button>

1 let inputs = document.querySelectorAll('inputs');
2 let btn = document.querySelector('#btn');
3 let sum = 0;
4
5 btn.addEventListener('click', function() {
6   for (let input of inputs) {
7     sum += Number(input);
8   }
9
10  console.log(sum);
11 });
```

№14

⊗jsPmDmMst

Numbers are entered into the first two inputs. By clicking on the button, the sum of these numbers should be displayed in the third input:

```
1 <input id="inp1">
2 <input id="inp2">
3 <input id="inp3">
4
5 <button id="btn">click</button>

1 let btn = document.querySelector('#btn');
2 let inp1 = document.querySelector('#inp1');
3 let inp2 = document.querySelector('#inp2');
4 let inp3 = document.querySelector('#inp3');
5
6 let sum = inp1.value + inp2.value;
7
8 btn.addEventListener('click', function() {
9   inp3.value = sum;
10 });
```

№15

⊗jsPmDmMst

A number is entered into the input. Upon loss of focus, the sum of the digits of this number should be displayed in the console:

```
1 <input id="inp">

1 let inp = document.querySelector('#inp').value;
2
3 inp.addEventListener('blur', function() {
4   let digits = +inp.split('');
5   let sum = 0;
6
7   for (let digit of digits) {
8     sum += digit;
9   }
10
11   console.log(sum);
12 });
```

Working out the studied material for work with the DOM

№1

⊗jsPmDmPm

Given an input and a paragraph. When focus is lost on an input, write the value of the input to the end of the paragraph text.

№2

⊗jsPmDmPm

Given several inputs, a paragraph and a button. By pressing the button, get the numbers in these inputs and write their sum in a paragraph.

№3

⊗jsPmDmPm

Input is given. A number is entered into it. After losing focus, find the sum of the digits of this number.

№4

⊗jsPmDmPm

Input is given. Enter numbers separated by commas in it. By loss of focus, find the arithmetic mean of these numbers (the sum divided by their quantity).

№5

⊗jsPmDmPm

Given 4 inputs. In the first input, enter the full name separated by a space. When focus is lost, write down the last name, first name and patronymic in the remaining 3 inputs.

№6

⊗jsPmDmPm

Given an input. The full name is entered into it, separated by a space. Full name is entered with a small letter. Make it so that when the input loses focus, the entered last name, first name and patronymic will automatically become capitalized (in the same input).

№7

⊗jsPmDmPm

Given an input. Text is entered into it. By losing focus, find out the number of words in this text.

№8

⊗jsPmDmPm

Given an input. The date is entered into it in the format *31.12.2016*. When focus is lost, put this date in the format *2016-12-31* in the same input.

№9

⊗jsPmDmPm

Given an input. The word is entered into it. By clicking on the button, check that this word is read from the beginning and from the end in the same way (for example, madam).

№10

⊗jsPmDmPm

Given an input. A number is entered into it. Check by loss of focus that this number contains the number 3 inside it.

№11

⊗jsPmDmPm

Given a few paragraphs and a button. By clicking on the button, write down at the end of each paragraph its ordinal number.

№12

⊗jsPmDmPm

Links are given. Add to the end of each link its [href](#) in parentheses.

№13

⊗jsPmDmPm

Links are given. If the link starts with [http://](#), add an → (it's done like this: [→](#)) to its end.

№14

⊗jsPmDmPm

Several paragraphs with numbers are given. By clicking on any paragraph, write in it the square of the number that it contains.

№15

⊗jsPmDmPm

Given an input. The date is entered into it in the format 31.12.2016. By losing focus, find out the day of the week (in a word) that falls on this date.

№16

⊗jsPmDmPm

Given an input. Implement buttons +1, -1 that will increment or decrement the input value by 1. Make it such a way that this value cannot become less than zero.

№17

⊗jsPmDmPm

Given an input. Given paragraphs. Let the total number of clicks on these paragraphs be written in this input.

№18

⊗jsPmDmPm

There are divs on the page. They have a text. Go through all the divs and cut off the texts so that they become 10 characters long. And add an ellipsis to the end of the truncated texts.

№19

⊗jsPmDmPm

Given an input and a button. By pressing the button, generate a random string of 8 characters and write it to the input.

№20

⊗jsPmDmPm

Given an input and a button. The user enters some string into the input. By pressing the button, mix the entered characters randomly and write it back to the input.

№21

⊗jsPmDmPm

Given an input, a button, and a paragraph. The input is the temperature in degrees Fahrenheit. By clicking on the button, display the temperature in degrees Celsius in the paragraph.

№22

⊗jsPmDmPm

Given an input, a button, and a paragraph. A number is entered into the input. On pressing the button, print the factorial of this number in the paragraph.

№23

⊗jsPmDmPm

Given 3 inputs, a button, and a paragraph. The coefficients of the quadratic equation are entered into the inputs. By pressing the button, find the roots of this equation and display them in a paragraph.

